

组会报告发言稿

各位老师、同学下午好，今天我汇报的论文是 **From Prompts to Printable Models: Support-Effective 3D Generation via Offset Direct Preference Optimization**, 这篇文章是今年2月发表在IEEE上的。

一、研究背景与问题

当前 text-to-3D 模型只追求**视觉保真**，完全不考虑 **3D 打印可制造性**：

- 悬空结构多，必须加大量支撑
- 材料浪费、打印时间长、表面质量差
- 后处理优化、切片软件加支撑，都属于**事后补救**

本文目标：

在生成阶段直接让模型学会“少支撑、易打印”。

二、核心方案：SEG 框架

SEG = Support-Effective Generation

核心思想：

把**支撑多少**当作**偏好信号**，用偏好优化精调预训练扩散模型，让模型天生倾向生成低支撑结构。

三、关键公式

1. 评价指标：NSV 归一化支撑体积

$$NSV(\mathcal{M}, \mathcal{M}_{\text{sup}}) = \frac{V(\mathcal{M}_{\text{sup}})}{V(\mathcal{M})}$$

- $V(\mathcal{M}_{\text{sup}})$ ：支撑体积
- $V(\mathcal{M})$ ：模型自身体积

- **意义**：支撑占比，越小越好打印。

四、为什么要用偏好优化？

因为我们要让模型学到：

支撑少 → 更好 → 应该被优先生成

所以采用 DPO 系列偏好学习。

五、标准 DPO 公式

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{y, (x^w, x^l)} [\log \sigma(r_\theta(x^w) - r_\theta(x^l))]$$

- x^w ：赢样本（支撑少）
- x^l ：输样本（支撑多）
- r_θ ：模型对样本的评分
- **目标**：让 $r(x^w) \gg r(x^l)$ ，使损失趋近 0。

六、重点：为什么 DPO 不好？

1. 直观原因

在支撑优化任务里，奖励分布**非常差**：

1. NSV 不是 0~1，数值可大可小，分布倾斜
2. 大部分样本 NSV 非常接近，差值极小
3. 样本密集、梯度小，DPO 学不到有效信号

2. 数学推导：DPO 失效原因

设真实奖励差为：

$$\Delta r = r^{\text{true}}(x^w) - r^{\text{true}}(x^l)$$

在我们的任务中：

$$\Delta r \approx 0 \quad (\text{绝大多数样本差距极小})$$

代入 DPO：

$$\log \sigma(r_\theta(x^w) - r_\theta(x^l))$$

当差值很小时：

$$\sigma(0) = 0.5, \quad \log(0.5) \approx -0.69$$

损失几乎**固定不变**，梯度趋近 0，网络**不收敛、学不会**。

更严重的问题：

DPO 无法区分“微小差距”和“巨大差距”，一律用相同损失处理。

但我们的任务恰恰需要：

- NSV 差很小 → 轻轻学
- NSV 差很大 → 狠狠学

DPO 做不到，所以**必须用 ODPO**。

七、ODPO 为什么好？

1. ODPO 核心：加入动态偏移 Δo

原文公式：

$$L_{\text{ODPO}}(\theta) = -\mathbb{E} [\log \sigma(r_\theta(x^w) - r_\theta(x^l) - \Delta o)]$$

其中偏移量：

$$\Delta o = \alpha \cdot \log(r'(x^w) - r'(x^l))$$

- r' ：NSV 计算的真实奖励
- α ：缩放系数
- $f = \log$ ：单调放大差距，解决倾斜分布

2. 数学推导：ODPO 如何解决 DPO 缺陷

设：

$$\Delta r_{\theta} = r_{\theta}(x^w) - r_{\theta}(x^l)$$

$$\Delta r_{\text{true}} = r'(x^w) - r'(x^l)$$

ODPO 优化目标变为：

$$\max \quad \sigma(\Delta r_{\theta} - \Delta o)$$

代入 Δo ：

$$\Delta r_{\theta} - \alpha \log(\Delta r_{\text{true}})$$

关键结论：

1. 当 Δr_{true} 很小时

$\log(\Delta r_{\text{true}})$ 是负数 $\rightarrow -\Delta o$ 变大 $\rightarrow$ 等效**拉大边际**，让模型更容易区分。

2. 当 Δr_{true} 很大时

$\log(\Delta r_{\text{true}})$ 为正 $\rightarrow$ 强制模型**必须拉开更大分数差**，否则损失巨大。

3. 自动加权

- 差距小 $\rightarrow$ 软学习
- 差距大 $\rightarrow$ 强惩罚

完美解决**奖励分布倾斜、样本密集、梯度消失**问题。

一句话总结

DPO：所有人用同一套标准学，学不动。

ODPO：根据真实差距动态调整边际，学得稳、学得准。

八、SEG 完整实现流程

步骤1：构建偏好数据

1. 数据集：Thingi10k (可打印3D模型)
2. Cap3D 生成文本提示
3. 对每个提示生成多个 3D 模型
4. 仿真计算支撑 → 得到 NSV
5. 两两配对：NSV 小=赢, NSV 大=输

步骤2：支撑结构仿真

1. 按 45° 识别风险面 (悬空面)
2. 光线追踪求支撑高度
3. 四面体分解精确计算支撑体积
4. 输出 NSV 作为奖励

步骤3：LoRA + ODPO 精调 (核心)

1. 冻结预训练模型 TRELLIS
2. LoRA 注入 Transformer 注意力层 (Q/K/V/O)
 - 只训练少量参数
 - 不破坏原模型能力
 - 训练快、显存占用低
3. 用 ODPO 损失训练, 让模型偏好低 NSV 结构

步骤4：推理

输入文本 → 输出**少支撑、可直接打印**的 3D 模型

这张图完整展示了**面向3D打印友好的生成式3D模型优化框架**, 核心目标是让AI生成的模型**支撑体积更小、打印更省料、结构更稳定**, 全程分为3大核心阶段, 我给你拆解得清清楚楚:

一、(a) Data Sampling: 数据采样与模型初始化

核心作用：搭建训练数据与基础模型

1. 数据来源：Database: Thingi10k

这是3D打印领域经典的开源数据集，包含大量真实可打印的3D网格模型，用来给模型提供“什么是好的打印结构”的学习样本。

2. 数据准备：Sample & Caption

从数据集中采样3D网格（3D Mesh），并生成对应的文本描述（Text），构建“文本-3D模型”配对数据，作为模型的输入。

3. 基础模型：Sparse Flow Transformer + LoRAs

- 主干网络是**稀疏流Transformer**，负责高效处理稀疏3D网格数据，生成初始的3D模型；
- 用**LoRA（低秩适配）**做轻量化微调，在不改动大模型主干的前提下，让模型快速适配“3D打印友好”的生成任务，训练成本更低、效率更高。

4. 反馈闭环：后续优化的梯度会反向传播回这个模块，持续迭代优化模型的生成能力。

二、(b) Support Simulation: 支撑结构仿真（核心创新点）

核心作用：给生成的模型做“打印可行性体检”，计算奖励信号

这一步是整个框架的灵魂，把3D打印的物理约束融入AI训练，分为两个子流程：

1. Risky Faces Detection: 风险面检测

- 输入：模型生成的 Sampled Mesh（采样后的3D网格，图里的企鹅模型）
- 作用：识别模型中**打印时需要加支撑的“风险面”**
 - 几何判断标准：通常是与打印平台夹角小于45°的悬空面（图中红色标注的区域、右侧示意图的红色斜边），这些面打印时会塌陷，必须加支撑。
 - 输出：标记出所有需要支撑的风险面，为后续支撑计算做准备。

2. Support Structure Simulation: 支撑结构仿真

基于检测到的风险面，完整模拟3D打印的支撑生成过程，分3步：

- **(a) Risky face identification**：确认风险面的位置、范围，精准定位需要支撑的区域；

- **(b) Support Structure Projection**: 从风险面向打印平台投影，生成支撑的轮廓；
- **(c) Tetrahedral Decomposition**: 对支撑区域做四面体分解，精确计算支撑的体积、重量、耗材量。
- 最终输出：这个模型的**NSV（归一化支撑体积）**，也就是我们之前聊的“奖励信号”——NSV越小，模型打印越省料，质量越好。

1. 风险面与光线投射逻辑：

"For every face f in the mesh, calculate the dot product of its normal n with the gravity vector $g=(0,0,-1)$. If $n \cdot g < \cos(135^\circ)$ (assuming 45° overhang threshold), the face is risky. From the centroid of each risky face, cast a ray in direction g . We use a ray-tracing kernel [30] to find the first intersection point P_{int} . If P_{int} is on the build plate ($z=0$), the support height is z_{face} . If P_{int} is on the mesh itself (internal support), the support height is $z_{face} - z_{P_{int}}$. This per-face ray casting is robust to non-manifold geometry (holes, self-intersections) because it does not require a watertight solid to calculate volume."

2. 四面体分解核心原理：

"The volume under a triangular face projected to a plane forms a triangular prism. Computing the volume of an arbitrary prism directly can be numerically unstable if the top and bottom bases are not perfectly parallel. Decomposing the prism into three tetrahedra is a standard method in Finite Element Method (FEM) [31] and computational geometry to calculate volumes of irregular polyhedra exactly."

$$V_{prism} = V_{tet1} + V_{tet2} + V_{tet3}$$

where the volume of a tetrahedron with vertices a, b, c, d is $\frac{1}{6} |(a-d) \cdot ((b-d) \times (c-d))|$

3. 全流程总结：

"In essence, the support simulation S proceeds by (1) identifying risky faces, (2) locating their intersection points with either the print bed or the mesh surface, and (3) accumulating the volumes of all resulting tetrahedra."

该方法是**逐面独立的局部体积计算方案**，全程不依赖网格的全局拓扑与水密性，完整步骤分为5步：

步骤1：前置准备与风险面识别（逐面处理的基础）

1. 固定打印坐标系：默认打印方向为重力反方向，重力向量 $g=(0,0,-1)$ ，打印底板位于 $z=0$ 平面；

2. 设定行业通用自支撑阈值：FDM打印的最大自支撑角度为 45° ，对应数学判断条件为：三角面法向 n 与重力向量 g 的点积小于 $\cos(135^\circ)$ ；
3. 遍历网格所有三角面，标记满足上述条件的**风险面（需要支撑的悬空面）**，作为后续逐面处理的对象。

步骤2：逐面光线投射，确定支撑的几何边界

对每一个风险面，执行独立的光线投射计算：

1. 计算该风险面的形心，沿重力方向 g 发射一条射线；
2. 通过Embree光线追踪内核[30]，求解射线与场景的第一个交点 P_{int} ，分两种情况确定支撑边界：
 - 若交点落在打印底板（ $z=0$ ）：支撑高度为风险面形心的 z 坐标 z_{face} ，支撑底部为底板上的投影区域；
 - 若交点落在模型自身表面（内部支撑）：支撑高度为 $z_{face} - z_{P_{int}}$ ，支撑底部为交点所在的模型表面区域。

步骤3：构建支撑对应的三角棱柱

以风险面的三角形为顶面，将顶面的3个顶点沿重力方向投影到步骤2确定的支撑底部平面，得到3个对应的投影顶点，组成棱柱的底面三角形。

顶面三角形+底面三角形，共同构成了该风险面对应的**支撑三角棱柱**，也就是该悬空面所需的支撑结构的完整几何形态。

步骤4：棱柱分解为3个四面体，精确计算体积

论文明确，直接计算任意倾斜棱柱的体积会出现数值不稳定，因此采用有限元法（FEM）的标准精确解法：

1. 将步骤3得到的三角棱柱，**无重叠、无遗漏地分解为3个独立的四面体**；
2. 用四面体体积的解析解公式，分别计算3个四面体的体积：

对于顶点为 a, b, c, d 的任意四面体，体积为：

$$V_{tet} = \frac{1}{6} |(a - d) \cdot ((b - d) \times (c - d))|$$

其中 $\cdot$ 为向量点积， $\times$ 为向量叉积，绝对值保证体积为正；

3. 棱柱的总体积=3个四面体的体积之和，即该风险面对应的支撑体积。

步骤5：逐面累加，得到总支撑体积

遍历所有风险面，重复步骤2-4，将所有单面对应的支撑体积累加，最终得到整个模型所需的**总支撑体积** $V(\mathcal{M}_{sup})$ 。

同时，该方法可直接扩展用于计算模型本体的体积 $V(\mathcal{M})$ ，只需将处理对象从「风险面」替换为网格的所有三角面，最终得到论文核心评价指标**归一化支撑体积NSV**：

$$NSV(\mathcal{M}, \mathcal{M}_{sup}) = \frac{V(\mathcal{M}_{sup})}{V(\mathcal{M})}$$

1. **完全不要求网格水密/流形**：该方法是逐面的局部计算，仅依赖单个三角面的有效顶点坐标，不需要网格全局闭合、不需要区分模型的「内部/外部」、不需要满足“每条边恰好被2个面共享”的水密性规则；
2. **对劣质网格鲁棒性极强**：哪怕网格存在孔洞、非流形边、自相交等拓扑缺陷，只要单个三角面是非退化的（顶点不共线、不重合），就能稳定计算出有效体积，不会出现数值崩溃；
3. **数值精度稳定**：通过四面体分解规避了倾斜棱柱的直接体积计算误差，采用解析解公式，无近似误差，和商业切片软件Cura的计算结果皮尔逊相关系数 $r > 0.85$ ，精度得到了工业验证。

三、(c) ODPO Optimization: ODPO算法优化

核心作用：用偏好优化，让模型生成“打印更友好”的3D模型

1. **样本构造**：基于支撑仿真的NSV结果，给同一个prompt生成的多个模型打分，构造**偏好对**：
 - NSV小的模型 = 好样本（图中最上方的企鹅，支撑最少）
 - NSV大的模型 = 坏样本（图中最下方的企鹅，支撑最多）
2. **ODPO算法 (Optimized Direct Preference Optimization)**
这是改进版的DPO算法，我们之前详细聊过它的核心逻辑：
 - 不需要真实奖励数值，只靠“谁比谁好”的偏好序训练；
 - 针对NSV差距做动态加权：小差距自动拉大边际，大差距强制模型拉开分差，自动优化模型的生成偏好；
 - 目标：让模型越来越倾向于生成**支撑少、省料、易打印**的3D模型。
3. **梯度回传**：ODPO优化的梯度会反向传播回(a)的Sparse Flow Transformer+LoRA模块，持续迭代模型，形成完整的训练闭环。

四、全流程一句话总结（组会金句）

本框架以Thingi10k数据集为基础，用稀疏流Transformer+LoRA生成初始3D模型，通过风险面检测与支撑仿真计算打印友好性奖励（NSV），再用ODPO偏好优化算法引导模型生成支撑更少、更适合3D打印的3D模型，全程端到端闭环训练。

十一、总结

这篇论文真正把 **AI 3D 生成** 和 **3D 打印工程** 结合起来：

- 不再只“好看”，而是**好用、能打印**
 - 用支撑仿真提供偏好信号
 - 用 ODPO 解决偏好学习困难
 - 用 LoRA 实现高效精调
- 为数字制造、个性化生产提供了非常实用的技术路线。